

Assignment 1: Command Interpreter

C S 429, Spring 2025

Unique ID: 50815, 50820, 50825, 50830, 50835, 50840, 50845, 50850, 50855, 50860, 50865, 50870

Lead TA: Pranay Gosar, Oleg Pittman, Jack Holt

Assigned: Monday, 13 Jan 2025 00:00 CT

Week 1 due: Thursday, 16 Jan 2025 23:59 CT

Week 1 last possible hand in: Thursday, 16 Jan 2025 23:59 CT

Week 2 due: Thursday, 23 January 2025 23:59 CT

Week 2 last possible hand in: Saturday, 25 January 2025 23:59 CT

Week 3 due: Thursday, 30 January 2025 23:59 CT

Week 3 last possible hand in: Saturday, 1 February 2025 23:59 CT

Final submission due: Thursday, 6 February 2025 23:59 CT

Final submission last possible hand in: Saturday, 8 February 2025 23:59 CT

1 Introduction

The purpose of this lab is to get you started on the path to proficiency in the C programming language and the Linux toolchain. You will use this combination of programming language and toolchain for most future labs. The focus is to give you practice in the style of C programming you will need to be able to do proficiently, especially for the later assignments in the class. Some of the skills developed are:

- Explicit memory management using the `malloc` and `free` calls, which is very different from what you have used in Java.
- Creating and manipulating pointer-based data structures: trees, linked lists, and hash tables. While pointers in C are similar to references in Java, there are key differences between them that you need to internalize.
- Working with strings. There is no built-in `String` class in C.
- Implementing robust code that operates correctly with invalid arguments, such as `NULL` pointers.

As C is substantially different from Java both in abstraction and in tooling, Appendix C provides a brief summary of the major points of difference that cause the most headaches for students, in our experience.

This assignment is split into four weeks. The first week is dedicated to basic machine setup, where you will learn how to setup your UTCS public machine accounts that you will use for the remainder of the class.

Over the following three weeks, you will be writing an *interpreter* for a simple low-level programming language ASML¹ that can perform operations on integers and character strings. You may already have used interpreters for programming languages such as Python or Matlab. In context of this lab, an interpreter is simply a program that performs the actions specified by an input program, typically producing some output.

While this handout is long, it is complete. *Read it very carefully!* There is a lot of background knowledge you need to know to successfully complete this lab.

2 Logistics

You may use up to two late (slip) days for each of weeks 2–4 of this assignment. You are **NOT** allowed to use any slip days for week 1.

Start early enough to complete your work before the due date. Assume things will not go according to plan, and so you must allow extra time for heavily loaded systems, dropped internet connections, corrupted files, traffic delays, minor health problems, *force majeure*, etc.

Unless otherwise stated, every project in C S 429 is an individual project. Before you begin, please take the time to review and abide by the course policy on academic integrity at: <https://www.cs.utexas.edu/academics/conduct>.

All hand-ins are electronic. You can do this assignment on any machine you choose.

The testing for your code is automated with Gradescope, using OS and compiler configurations similar to those on the UTCS Linux machines. You are *heavily* encouraged to do all your work on these machines. For this and future assignments, you are allowed unlimited submissions on Gradescope.

3 Week 1: Setup

The following section lists the steps you must complete for the first checkpoint of this lab. These steps will guide you through setting up your environment.

Go through the following list of one-time setup actions and complete any that you do not have set up already. Do not proceed to the actual assignment until you have completed these actions. Ask a member of the teaching team if you get stuck on any of these steps.

You may already have completed some of these tasks during a previous semester. If so, you can skip those sections. As you set up your environment, check out [*this link*](#) if you want additional help.

3.1 UTCS Account

If you do not already have an account, get one following the directions given [*here*](#). The account will be necessary to log onto the lab machines, and is thus necessary for this class. There is often a delay of a day or two in setting up an account. If this delay impacts you, inform a member of the teaching team *immediately*.

You will typically be accessing and working on one of the UTCS Linux machines (the “remote machine” or “lab machine”) from your desktop or laptop machine (the “local machine”)². Your local machine can be running Windows, Mac OS, or Linux (the “local operating system”).

UT and UTCS require that you use certain security procedures to perform the access. Specifically, you will be connecting to the UT network over a virtual private network (VPN), and you will be logging in using the `ssh` (secure shell) command.

¹ASML stands for “A Simple Machine Language”.

²If you are running out of a VM on your local host, consider that VM to be the local machine.

3.2 UT VPN

If you are off-campus, you will need to use the UT VPN software to connect to the UT network. To set up the VPN, follow the instructions [here](#). You will need to install the VPN, and then connect using Duo, which can be set up [here](#).

Note that you will not need to use the VPN if you are on-campus and are connected to the `utexas` network (not the `utguest` network). If you've never used this network before, you just need your UT EID and password to log in, just as if you were logging in to Canvas.

3.3 SSH

Once the VPN is set up and you have a UTCS account, you will need access to a terminal application on your local machine in order to remotely access files and run commands on the lab machines from your local machine using `ssh`³. This terminal application will vary depending on your local operating system.

- On Windows 10, use Windows Terminal. This is a Microsoft application and can be installed from the [Windows Store](#). Once installed, open it, and the default terminal should be PowerShell. If not, click the “downward caret” at the top of the screen and select PowerShell. You can run the SSH command to get to the UTCS lab machines from here. To open another tab, click the “plus” icon at the top, next to the current tab.
- Windows Terminal should come preinstalled with Windows 11, but if not you can install it from the [Windows Store](#). Once installed, open it, and PowerShell should be the default terminal that opens. If not, click the “downward caret” at the top of the screen and select PowerShell. To open another tab, click the “plus” icon at the top, next to the current tab.
- On Mac OS or Linux, use the preinstalled terminal application.

Once you have your terminal figured out, start it up. You will see something like this in the terminal window:

```
(base) sidchat@sidchats-MacBook-Pro ~ %
```

This is called the *terminal prompt* (or just “prompt”). When needed, we will generically indicate this prompt as `prompt>` (for local machines) or `linux>` (for lab machines). Sometimes, we (or other online resources) may also refer to this as `$`. *Remember that the command you type is only what follows the prompt.*

Select one of the lab machines from the list found [here](#). This list is updated in real-time. Make sure to select a machine that is listed as being up and having a low load. The file system is shared across all lab machines, so you can switch machines at any time without any concern. (You may need to do this, for example, if a machine goes down or the load on it gets too high.) As a best practice, you may want to select a primary machine (the one that you normally use) and one or more secondary machines (that you would switch to if your primary machine is down, disconnected, overloaded, etc.). The files in your home directory are shared between all public CS lab machines, so you will not need to copy files between machines in order to use multiple machines. Then type the following command at the *local machine prompt*: `ssh [ID]@[HOSTNAME].cs.utexas.edu`, where `ID` is your UTCS login id and `HOSTNAME` is the name of the lab machine⁴. You will be prompted for your UTCS password; once this is entered, you will see a

³`ssh` stands for “secure shell”.

⁴Our typographical convention is that anything enclosed in a box `THUS` is a parameter that is to be replaced by a value. So, for user `sidchat` using the host named `chess`, the actual command entered would be `ssh sidchat@chess.cs.utexas.edu`.

```

Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-155-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage
* UTCS FAQ: http://www.cs.utexas.edu/facilities/faq/
* Report software problems to help@cs.utexas.edu
System information as of Sat 19 Aug 2023 09:08:22 AM CDT

System load:  0.0           Temperature:           29.8 C
Usage of /:   50.9% of 46.63GB Processes:             282
Memory usage: 8%           Users logged in:       0
Swap usage:   2%           IPv4 address for enp0s31f6: 128.83.144.18

This network and computing environment (including this system) is
owned and managed by the UT Austin Department of Computer Science,
and intended for instructional, research, and administrative use by
authorized users only. By using this system, all users acknowledge
notice of, and agree to comply with, the UT Austin Acceptable Use
Policy (https://security.utexas.edu/policies/aup). Unauthorized,
improper, or negligent use of this environment may result in
administrative disciplinary action, civil charges/criminal
penalties, and/or other sanctions as set forth in University or
Department policies.

Last login: Wed Apr 19 21:19:18 2023 from 072-177-088-139.res.spectrum.com
chess$

```

Figure 1: Welcome message on UTCS Linux machine.

welcome message similar to the one shown in Figure 1 on your (local) terminal window. You are now (finally!) sshed into the lab machine. That `chess$` on the last line is the “remote prompt”.

3.4 SSH Keys

Once you have sshed onto a remote machine, you should set up your **ssh keys** so you can access the lab machines *without needing to enable the VPN*. This step is not strictly necessary, but it makes life a lot simpler, so we strongly recommend that you do it. Follow the instructions [here](#). Make sure you select the correct operating system from the Mac, Windows, and Linux tabs.

3.5 GitHub

You will need a GitHub account for this class. If you don’t already have one, sign up for a free account [here](#).

3.6 VSCode

VSCode is a code editor that we will be using this semester. **It is not required; you can use a different IDE if you are proficient in it.** However, we like VSCode for its ability to directly access and edit remote files, such as the ones located on the lab machine, without needing to copy them to and from your local machine. Here are the links to download *VSCode*, set up the *remote development* extension, and install an *extension* to provide language support for C within VSCode.



Figure 2: Representative screenshot showing remote machine access.

We will be using VSCode as a *code editor* rather than as a full-fledged IDE. This means that you will be expected to be able to perform actions such as compiling programs, debugging programs, and performing version control operations from the Linux shell prompt. These skills will be tested in the mastery requirement.

4 Week 1: What to Do

With all that setup in place, we can finally get to the week 1 assignment. There are just two things that you need to do.

1. Open a terminal window on your local machine and `ssh` into your account on any of the UTCS servers. At the remote prompt, type the command `whoami`. Then type the command `logout`. Take a screenshot that clearly shows the terminal, the local command that you typed in, the remote prompt, the two remote commands (`whoami` and `logout`), and the final local prompt. Figure 2 shows a representative screenshot. Save this screenshot as `screenshot1.png`.
2. Using your Web browser, log into your account at `github.com`. Click the circle at the top right-hand corner of the page that contains your picture or avatar, and then select `Your Profile` from the dropdown menu. Take a screenshot. Figure 3 shows a representative screenshot. Save this screenshot as `screenshot2.png`.

5 Week 1: Handing in Your Work

Hand in this checkpoint by uploading your two screenshots in Canvas.

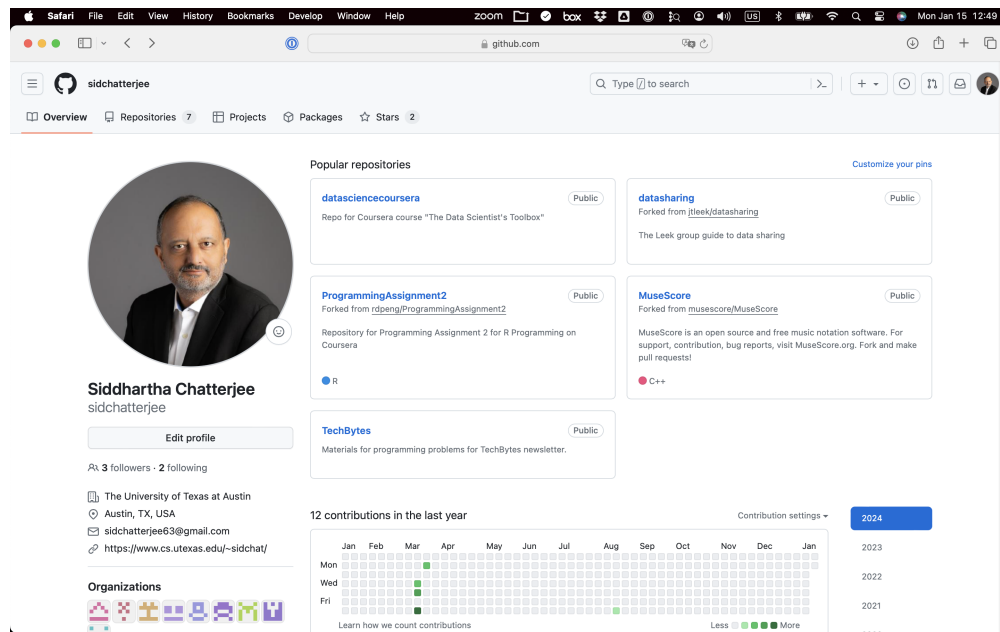


Figure 3: Representative screenshot showing GitHub account.

6 Week 1: Evaluation

This checkpoint is worth 1% of the total course grade. Each screenshot is worth 0.5 points. We will grade them manually to make sure that all the necessary components are present. If in doubt, consult a member of the teaching team before uploading.

7 Weeks 2–4: Download and Setup

With week 1 out of the way, we can now begin the programming portion of the assignment.

First, go to our GitHub Classroom page by clicking on <https://classroom.github.com/a/v-kHKywj> and click the “Accept this assignment” button. This will automatically create an assignment repository specific to you on GitHub. (Let’s say the URL for this created repository is *your-repo-url*.) Now, SSH into a UTCS lab machine and clone this created repository from the Linux shell prompt using the command `git clone [REPO-URL]`. Remember to change `[REPO-URL]` to *your-repo-url*, the URL for your personal copy of the repository. Ask a TA for help if you are having trouble with these steps.

8 Weeks 2–4: Background

As a very basic overview of the theory underpinning this lab, a program written in a programming language is processed in your computer through a multi-step *translation and execution process*.

- The source code that you write and store in a file is simply a sequence of characters (a `String` object in Java, if you will). Such a file, containing part or all of your program, is the input to this process. As

a running example, let's suppose this input string is $P = \text{"add_x0_x1_x2; _print_x0"}$, which contains 22 characters. (Don't count the double-quotes. Do count the five whitespaces, shown as `_`.)

- The first step (lexical analysis, aka “lexing”) breaks this input sequence into a sequence of *tokens*, which are the smallest units of syntax in the language. In our example, the sequence after lexing is $T = [\text{"add"}, \text{"x0"}, \text{"x1"}, \text{"x2"}, \text{";"}, \text{"print"}, \text{"x0"}]$. This is now a sequence of seven tokens, and the whitespace characters have been discarded.
- The next step (syntactic analysis, aka “parsing”) takes the sequence of tokens and organizes them into some internal representation, often a list or a tree, that reveals the intended structure of the program. Structure is driven by the *grammar* of the programming language. In our example, the first four tokens form one command (which adds the values of variables `x1` and `x2` and stores it in variable `x0`) and the last two form a second command (which prints the value of variable `x0`). So the internal representation after parsing might be a linked list of these two commands, with the details of each command being packaged as a record. Concretely, the internal representation might look like this: $IR = [\text{command}(\text{add}, \text{x0}, \text{x1}, \text{x2}), \text{command}(\text{print}, \text{x0})]$. The token corresponding to the semicolon character is no longer needed; it was used simply to separate the first command from the second, and that function is now accomplished by the combination of packaging each command into a record and maintaining their relative order through the linked list.
- There may be further steps at this point, such as semantic analysis and code generation, resulting in the production of an *object module*. A `.class` file in the Java world is an example of an object module. If needed, multiple object modules may be combined into an *executable*.
- Finally, the computer executes the executable by *interpreting* the entries in it. This involves using this internal representation to execute the program's instructions and produce the desired output.

The ASML language is very simple compared to high-level programming languages such as Python, Java, or C, but is just as expressive⁵. The only parts of the process above that are needed are lexing, parsing, and interpreting. The lexer is already implemented for you. Your goal in this lab is to implement the parser and interpreter for ASML while handling any errors that might arise in doing so, such as malformed commands and incorrect command arguments.

To keep your work manageable, we have structured the work in weekly stages, with each stage building on top of the previous ones.

- **Week 2:** You will parse and interpret programs containing some core operations on integers and straight-line control flow (i.e., the next command to be executed is the one that follows the current command in the intermediate representation).
- **Week 3:** You will continue by parsing and interpreting bit operations such as shifts and bitwise logical operations like *and* and *or*. Control flow is still straight-line.
- **Week 4:** You will add control flow consisting of branches and function calls. A branch is a way to control the execution flow of your program, and can either be conditional (branch to a label only if some condition is true) or unconditional (always branch to a label). This will include implementing your own hashmap (C doesn't have a built-in one!) for figuring out which commands to run next based on the label associated with the current instruction (the branch). Function calls also depend on this, where the label acts as the function call name.

⁵This means that anything that you can write in those languages can also be written in ASML, albeit with a lot more effort.

8.1 Details of the language

The ASML programming language is particularly simple and flat in structure.

- There are 32 predefined *variables*, named `x0` through `x31`. Each of these variables is a 64-bit vector; smaller widths of these variables may be accessed. They are *mutable*, i.e., their values can change while running a program.
- There is a fixed amount of mutable *memory*, which you should think of an array of bytes. Each byte is readable and writable.
- Numerical *constants* (aka “integers” or “immediates”) are numbers specified either in base-10, base-16, or base-2. A base-16 constant starts with the prefix `0x`, while a base-2 constant starts with the prefix `0b`. Constants are (obviously) immutable. A valid constant is representable as a 64-bit vector.
- There are 24 predefined *commands*. A command specifies an action to be taken. The remaining arguments of a command depend on this action. The details of the commands are give in Table 1.
- *Strings* are NUL-terminated arrays of `chars`. A string can be used in a program only as an argument to a `put` command.
- *Labels* are strings that attach to commands and serve to identify them for branching and calling purposes. They cannot be used for any other purpose.
- A *program* is a finite sequence of commands, with commands being separated with either newlines or semicolons. Execution of a program starts with the first command of the sequence, and continues from there according to the control flow until it reaches the end.

Week	Description	Syntax	Effect	Example
2	Add one value to another	add <var0> <var1> <var2>	<var0> = <var1> + <var2>	add x0, x1, x2
2		add <var0> <var1> <num>	<var0> = <var1> + <num>	add x0, x1, 42
3	Bitwise AND of values	and <var0> <var1> <var2>	<var0> = <var1> & <var2>	and x0, x1, x2
3	Arithmetic shift right	asr <var0> <var1> <num>	<var0> = <var1> >> <num>	asr x3, x4, 14
4	Unconditional branch to label	b <label>	See Note 1 below.	b L3
4	Conditional branch to label	b.<cond> <label>	See Note 2 below.	b.ge L3
4	Call function at label	call <label>	See Note 3 below.	call Fun
2	Compare two values (signed)	cmp <var1> <var2>	Set is_greater, is_equal, and is_less flags based on <=>	cmp x1, x2
2		cmp <var> <num>		cmp x3, 0
2	Compare two values (unsigned)	cmp_u <var1> <var2>	Set is_greater, is_equal, and is_less flags based on <=>	cmp_u x1, x2
2		cmp_u <var> <num>		cmp_u x3, 0
3	Bitwise XOR of values	eor <var0> <var1> <var2>	<var0> = <var1> ^ <var2>	eor x0, x1, x2
3	Load data from memory into variable	load <var0> <num> <var1>	See Note 4 below.	load x0, 4, x1
3		load <var0> <num> <num1>		load x0, 8, 512
3	Logical shift left	lsl <var0> <var1> <num>	<var0> = <var1> << <num>	lsl x3, x4, 14
3	Logical shift right	lsr <var0> <var1> <num>	<var0> = <var1> >> <num>	lsr x3, x4, 14
2	Assign a value	mov <var> <num>	<var> = <num>	mov x5, 0x4ad0
3	Bitwise OR of values	orr <var0> <var1> <var2>	<var0> = <var1> <var2>	orr x0, x1, x2
2	Print value in specified format	print <var> <base>	Prints value in specified base	print x15, d
2		print <num> <base>	Prints number in specified base	print 0xabcd, d
3		print <var> s	Prints string from contents of <var> treated as memory address	print x5, s
3		print <num> s	Prints string from given memory address	print 0xabcd, s
3	Store string in memory	put <str> <var>	See Note 5 below.	put "Hello", x5
3		put <str> <num>	See Note 5 below.	put "Hello", 0xabcd
4	Return from function call	ret	See Note 6 below.	ret
3	Store data into memory from variable	store <var0> <var1> <num>	See Note 7 below.	store x0, x1, 4
3		store <var0> <num1> <num>		store x0, 512, 8
2	Subtract one value from another	sub <var0> <var1> <var2>	<var0> = <var1> - <var2>	sub x0, x1, x2
2		sub <var0> <var1> <num>	<var0> = <var1> - <num>	sub x0, x1, 42

Table 1: List of ASML commands, alphabetically by action.

Notes:

1. Set following command to the one tagged by <label>.
2. If <cond> is true, set following command to the one tagged by <label>. Otherwise, set following command to next command of current one in the linked list.
3. Save values of x0 through x31 and the pointer to the next command in a StackRecord. Push this StackRecord on the interpreter's stack. Set following command to the one tagged by <label>.
4. Load <num> bytes of memory in little-endian byte order from the starting memory address specified by <num1> or contents of <var1> into variable <var0>, with zero-fill if needed. <num> ∈ {1, 2, 4, 8}.
5. Store <str> into memory, including a terminating NUL, starting at the memory address specified by either by the contents of <var> (interpreted as a memory address) or by <num>.
6. If there is a StackRecord on the interpreter's stack, load the values of x1 through x31 and the pointer for next from said record. Also remove that record from the stack. Otherwise cause interpretation to end without error.
7. Store <num> bytes of data in little-endian byte order from variable <var0> into memory, starting from the memory address specified by <num1> or contents of <var1>. <num> ∈ {1, 2, 4, 8}.

8.2 Processing ASML: Lexing, Parsing, Interpreting

Let's go back to our running example of an ASML program: $P = \text{"add_x0_x1_x2; _print_x0"}$. In the following three subsections, we'll explore how this code is turned into our expected output and what you need to do to get there.

8.2.1 Lexing

Lexing is done for you, but understanding the basics about what it does will help you understand what parsing and interpreting do. Lexing is the process of taking our input source code and “chunking” it into a sequence of *tokens*, which are the smallest meaningful building blocks of our language. For the program P above, this would produce the following sequence of seven tokens:

$T = [\text{TOK_ADD}, \text{TOK_IDENT}, \text{TOK_IDENT}, \text{TOK_IDENT}, \text{TOK_NL}, \text{TOK_PRINT}, \text{TOK_IDENT}]$.

This list only shows the *token types* of these tokens. There are many types of tokens that we use in this lab: one for every command (TOK_ADD and TOK_PRINT), one for variables (TOK_IDENT), one for numbers (TOK_NUM), and some others that we use to facilitate lexing and parsing on our side (TOK_NL). See the file `include/ci/token_type.h` for a listing of the valid token types for ASML.

Lexing also handles a few housekeeping tasks, such as skipping over comments and handling alternate forms of commands⁶. Note, however, that ASML is case-sensitive, so writing `ADD` instead of `add` won't work.

The lexer has a very limited interface to the other components of the system.

- A lexer instance is initialized by calling `lexer_init()`. This is already done for you in the function `run_file` in `src/ci/ci.c`.
- The function `lexer_next_token()` returns the next token. This is a variable of the user-defined data type `Token`, which we discuss in detail below in Section 8.2.2. The parser you write will be calling this function repeatedly.
- The function `print_lexed_tokens()` is a helper routine that prints all the lexed tokens in the input file.

See the file `include/ci/lexer.h` for details of these functions such as parameter and return types.

8.2.2 The Token data type

Since this is your first encounter with programming in C, we thought that examining a `struct` in detail would prove to be instructive. (For the present, you can think of a `struct` as C's simpler analog of an object in Java. This is true, but not the whole truth.)

Open up `include/ci/token.h` and you will see the following definition of the `Token` data type.

```
typedef struct {
    TokenType    type;        // The type of this token.
    const char *lexeme;       // Pointer to the start of this token in the source text.
    int          length;      // The length of the token.
    int          line;        // The line number where this token is located (1-based).
    int          column;      // The column number where this token starts (1-based).
} Token;
```

⁶The command `add x0 x1 x2` can also be written as `add x0, x1, x2`, for reasons that will become clear later in the semester. The lexer handles this alternate form by skipping over the commas.

The parser is not interested in the line or the column fields; the lexer will set those up, and they are primarily used for error handling. What the parser cares about are the first three fields:

- `type`: The token's type, as defined in `include/ci/token_type.h`. Your parser will use this information to decide what to do with this token.
- `lexeme`: The pointer to the start of the token within the original string.
- `length`: The length of the token in chars.

Here is the token sequence T in detail, showing the first three fields of each token.

```
[{TOK_ADD, 0, 3}, {TOK_IDENT, 4, 2}, {TOK_IDENT, 7, 2}, {TOK_IDENT, 10, 2},  
{TOK_NL, 12, 1}, {TOK_PRINT, 14, 5}, {TOK_IDENT, 20, 2}].
```

Note that the `lexeme` field does not store the actual text of the token, but only identifies its starting position within the original string — a *pointer*. `TOK_IDENT` is a special token that corresponds to any *identifier* in the code. For the purposes of this lab, an identifier is any token that does not correspond to a command, number, or reserved symbols like colon and semicolon.

8.2.3 Parsing

Parsing is the first thing you will do yourself, by filling in code in the file `src/ci/parser.c`. The parser examines the tokens provided by the lexer and outputs a linked list of commands to be executed by the interpreter.

The user-defined data type `Command` in the file `include/ci/command.h` describes the various fields of a command. It references several other user-defined data types, all of which are defined in the same file. You should be able to understand most of the fields without difficulty. The only type that is unfamiliar is `Operand`, which is a union. Such a type can hold one (and only one) of any of its three variants (in this case, `num_val`, `string_val`, or `base`) at any point of time.

When the parser looks at the first token, it sees that it has a `type` of `TOK_ADD`. This tells it to expect three additional tokens to fill out the command: a destination and two input operands. Referring to Table 1, we see that the next two tokens should be of type `TOK_IDENT`, but the type of the third token can be either `TOK_IDENT` or `TOK_NUM`. It now obtains these tokens one after one from the lexer, and can fill out the details of this `Command`. Continuing with our example program P (which by now the lexer has converted to the token sequence T), the parser finally produces the following list of commands IR that the interpreter then consumes:

```
[{CMD_ADD, ..., <0||>, <1||>, <2||>, false, false, false, false, BRANCH_NONE},  
{CMD_PRINT, NULL, <0||>, <||>, <||>, false, false, false, false, BRANCH_NONE}].
```

8.2.4 Interpreting

Interpreting, the final part of ASML processing, is implemented by the code in `src/ci/interpreter.c`. An interpreter is initialized with an empty stack and with all 32 variables assigned 0. Interpretation of a linked list of commands always begins with the interpretation of the first command in said linked list. Interpreting a command has two parts:

1. *Performing the action specified by the command.* This is generally straightforward: The interpreter fetched the `Command`, decodes it based on the `CommandType` and the types of the operands, and

performs the action. If a command is of type `CMD_ADD`, it determines whether the operands are variables or numbers, retrieves the appropriate values, performs the addition, and stores the result in the destination variable.

2. *Selecting the following command to be interpreted.* By default, the command to be interpreted after the current command is its successor in the linked list of commands, i.e., the command pointed to by its `next` field. The only exceptions to this default rule are commands of type `CMD_BRANCH`, `CMD_CALL`, and `CMD_RET`. Since these three commands appear only in week 4, we defer the details until Section 8.3.3.

There are only two situations that will cause termination the of interpretation of a valid (errorless) ASML program:

1. A `ret` is processed by an interpreter with an empty stack.
2. The end of the source file is reached, this causes termination regardless of the state of the interpreter's stack

8.2.5 Error handling

We may not always be given well-formed programs. Consider these examples: `ADD x0, x1, x2;` `add x0, 42, x1;` `add 30, x0, x1;` `add x50, x10, x11;` `add x0, x1, 0b0123;` and `add x0, x0.` Each one has an error that makes it malformed (syntactically invalid) or meaningless (semantically invalid). *Stop and take time to figure out exactly what the errors are.* What should we do in such cases? This gets us into the thorny area of error handling.

Given the multi-step nature of the interpretation process, errors can happen in multiple places. The principle of error handling you will use is this: *Handle the error as early as you can in the process, but no earlier.* Typically, these failures means that no later stages can be executed meaningfully, so you will print an error message and return an appropriate indicator to your caller routine.

We classify errors by the earliest time that we can recognize them. Here are the classes.

1. *Lexical errors* are those that cause lexical analysis to fail to create a valid token, e.g., an unrecognized character or an invalid syntax for an identifier. The supplied lexer module handles such errors for you.
2. *Syntactic errors* are those that cause the parser fail to create a valid command from a sequence of token.

If at any point the parser sees a token that it cannot handle, it must immediately halt and do the following:

- (a) Set the `had_error` field to `true` to signal that it found a problem.
- (b) Return any commands it managed to parse successfully up to that point.

Important! The parser must not move past the problematic token! This means that if you were parsing a command like `add` and you encounter a number `0o777`, which could be a valid octal number but is not in any of our supported bases, the parser must stop with its *current* field pointing to the token `0o777`.

You will know whether you implemented error handling correctly if you pass the variety of tests specifically designed to test the behavior of the parser in the presence of errors.

It is an error to have multiple commands on the same line unless they are separated by a semicolon (;). It is your job to enforce this behavior. Once again, you will know if you did this correctly by passing the appropriate tests.

3. *Interpretation errors* are those that fail to interpret a command meaningfully. We will discuss the specifics of these situations and the appropriate actions in Sections 8.3.2 and 8.3.3.

8.3 What you will code

The only four files that you will be modifying in this assignment are `parser.c`, `interpreter.c`, `command.c`, and `label_map.c`. All of these files are in the `src/ci` directory.

Study the header files in the `include/ci` directory carefully. They provide a significant amount of documentation and examples to learn how to do the lab. We *heavily* encourage you to avoid writing any code before fully understanding the relevant pieces for the current checkpoint you are working on.

When you are ready to write your own code, look for the `STUDENT: TODO` comments. These tell you where you are expected to make changes and (briefly) what you're supposed to be doing.

There is some new terminology used in the lab and the following sections. `dest` refers to the destination, the variable where the result of the command will be stored. In all commands that have a destination, this is the first operand. `val_a` and `val_b` refer to the two operands a command may take in. Not all commands have a `val_b`, but it is always the third operand while `val_a` is always the second. In the code, an immediate is synonymous with a number.

We provide some well-documented helper functions that you can use. Keep in mind that these are suggestions as to how to approach the problem and you are not required to use them, but using these will significantly help reduce code duplication. **Note:** Not all of the helper functions we give you are going to be filled out; it is your job to fill them out if you wish to use them.

There are many small helper functions in this file that are used on our side, but here are the four you will be primarily working with in `parse_cmd()`, where the brunt of the parsing work is done:

- `parse_variable_operand`
- `parse_imm`
- `parse_var_or_imm`
- `parse_base.`

Read the code and comments to figure out when/how to use these. You need to implement these yourself and you should use the other helper functions we provide to simplify their implementations.

8.3.1 Week 2

For this week, you will be implementing the following commands: `add`, `sub`, `mov`, `cmp`, `cmp_u`, and the numerical cases for `print`. See the details of these commands in Table 1.

Figuring out a good starting point may be a tad bit overwhelming. We suggest the following order of implementation:

- In `parser.c`:
 1. Implement `parse_variable_operand`.

2. Implement `parse_imm`.
 3. Add the case to handle `TOK_MOV` in `parse_cmd`.
- In `interpreter.c`:
 1. Add the case to the switch statement in `interpret` to handle `CMD_MOV`.
 2. Update the interpreter to move to the next command if performing the action specified by the command didn't cause any errors.

You should then be able to pass the `mov` tests for week 2 and can move on to handling other commands and implementing any other functions.

We once again stress the liberal use of GDB. Not only is it a good tool for debugging, but it can also clear up any confusion about function arguments or runtime values you aren't sure of! We'll provide a more in-depth guide to using GDB on Ed Discussion. Furthermore, we *highly* encourage the use of *Test Driven Development* (TDD)! Don't get overwhelmed with trying to do everything all at once — it's best to start with the simplest test cases, figure out why those are failing, and fix them one at a time, without worrying about the entire implementation. Using GDB will be extremely helpful when paired with this technique.

Handling memory cleanup When you pass all the tests, implement the `free_command` in `command.c` to clean up memory allocated for commands to be done with week 2. You do not need to modify anything in `command.c` for the rest of the lab.

8.3.2 Week 3

For the third week, you will be implementing the following additional functionality:

- The bitwise logical commands `and`, `eor`, and `orr`.
- The shift commands `lsl`, `lsr`, and `asr`.
- The case for `print` to handle printing strings.
- The memory commands `load`, `store`, and `put`.

See the details of these commands in Table 1.

Implementing the memory commands will require interacting with the memory subsystem through the helper functions we provide, `mem_load()` and `mem_store()`. These functions are defined in `include/ci/mem.h` and implemented in `src/ci/mem.c`.

The `store` and `put` commands do similar things in that they both store data at a specified memory address. However, `store` stores the contents of a variable (or a subsection of a variable), while `put` stores a string literal, which can be of arbitrary length. In addition, `put` includes a terminating NUL byte at the end of the string. Internally, they are both implemented with `mem_store()`, but you must store all the bytes of the string (rather than just a fixed number of bytes for a variable) when implementing `put`. The `load` command goes the other way, loading a specified number of bytes from memory into one of the 32 variables.

There are some constraints on what can be loaded and stored. The functions `mem_load()` and `mem_store()` return `false`, signalling that something went wrong, if one of three things happen.

- The number of bytes you are trying to manipulate at one time is not one of 1, 2, 4, or 8.

- the source/destination memory address is `null`.
- The memory space being manipulated goes beyond the length of the memory array. The latter happens when the address plus the number of bytes you want to access goes beyond the maximum size, in bytes.

These errors need to be handled in your code by setting the `had_error` field of the interpreter to `true` whenever they occur.

8.3.3 Week 4

In week 4, you will be implementing *control flow*, i.e., those situations where the instruction to be executed following the current one is (or may be) different from its linked-list successor. This happens only for four commands: `b`, `b.<cond>`, `call`, and `ret`. The two concepts we need to implement control flow correctly in the interpreter are *labels* and *the interpreter stack*.

- A *label* is an identifier that tags the command immediately following it in the source code. Its only purpose is to provide a symbolic way to identify that command, and it may only be used as the argument of the `b`, `b.<cond>`, and `call` commands. A label is of type `TOK_IDENT` and must be immediately followed by the colon character (`:`, of type `TOK_COLON`). Furthermore, a label is not required to be on its own line; a command might follow it, like `foo: add x0, x1, x2`.

Since the purpose of a label is to identify an instruction, it makes sense to organize them as a dictionary of key-value pairs, with a label being a key and a pointer to the command it tags being the value. You will do this by implementing the hash table defined in `include/ci/label_map.h` by completing the code in `src/ci/label_map.c`. Refer to *Mike's slides* from CS 314 if you need a refresher on how they work. You do not need to implement table resizing.

- The *interpreter stack* is a stack data structure whose elements are of type `StackEntry`. You can find it in the definition of the `Interpreter` type in the file `include/ci/interpreter.h`. A `StackEntry` is similar to the stack frame that you learned about in CS 314. As with stack frames, the purpose of a `StackEntry` is to allow a function call/return protocol to be implemented⁷.

The `b` and `b.cond` commands at the source level are both mapped to the single `CommandType` `CMD_BRANCH` in the interpreter. The `branch_condition` field within the command specifies the condition under which the branch should be taken (i.e., the following instruction should be the one tagged by the label). If the condition doesn't hold, the branch is not taken (i.e., the following instruction is the linked-list successor of the current instruction).

The data type representing the interpreter (the `Interpreter` type in `include/ci/interpreter.h`) contains three Boolean fields named `is_greater`, `is_less`, and `is_equal`. These fields are set by the `cmp` and `cmp_u` commands. Use these truth values to figure out whether or not the condition specified in the `CMD_BRANCH` holds.

A function in ASML is quite primitive.

- A function is simply a sequence of instructions starting with a labeled one, the label being the name of the function.
- A function is invoked with the `call` command.

⁷What exactly is a function in ASML? Don't worry about it now. Go with the flow, and we'll get to it in week 9 of the semester.

- A function returns to its caller by executing a `ret` command, which has no arguments.
- If a function takes any arguments, those arguments are supplied in successive variables starting from `x0` (i.e., first argument in `x0`, second argument in `x1`, and so on).
- If a function returns a result, it does so in variable `x0`.

Since ASML's function model is so primitive, the `call` and `ret` commands need to do some extra work. The `call` command first saves the values of the 32 variables `x0` through `x31` and a pointer to the linked-list successor of the `call` command in a `StackEntry`, then pushes this to the interpreter's stack, and finally sets the following instruction to be the one tagged by the label. The `ret` command undoes this stacking. It pops the interpreter's stack, restores the values of variables `x1` through `x31`, and finally sets the following command to the one in the `StackEntry`. Interestingly, this is sufficient to correctly handle recursive functions as well.

For this week, you will need to implement the necessary TODOs in the files `src/ci/parser.c` and `src/ci/interpreter.c`, as well as the hash table in `src/ci/label_map.c`.

Handling memory cleanup Since a hash table is by nature a dynamic data structure, you will need to use `malloc` and friends to reserve and cleanup memory. Also, depending on your implementation, you will likely need to be mindful of freeing the labels when parsing after you put them into the map. Finally, give some thought as to how to free the interpreter's stack, both as you traverse through it and in cases of errors during interpreting.

You do not need to call functions dealing with freeing the label map; we do so for you. You do, naturally, have to implement them, however.

8.3.4 Running your code

In order to build an executable that you can run, type the command `make all` at the Linux prompt from your project directory. This will run the C compiler `gcc` on the necessary files and put everything together into a binary file called `ci`, that lives in the `bin` subdirectory. We have also provided you a pre-built reference implementation of the interpreter, which is in the binary file `ci_reference`. You can run either of these binaries in two modes.

- Your primary mode should use the following command line at the Linux prompt from your project directory: `bin/ci -i foo -o bar` or `ci_reference -i foo -o bar`. This will read the input program from a text file called `foo` and write output to a text file called `bar`. If you omit the `-o bar` part, any output will be written to your terminal window instead.
- A secondary mode is to run the program in interactive mode, which is similar to the way you use Matlab or Python. To do this, run the executable with no flags (i.e., type either `bin/ci` or `ci_reference` at the Linux prompt). You will be given a prompt (`CI>`) where you can write commands. The syntax for this is slightly different than what you see in the testcases. We automatically end the testcase and execute the commands you have written the moment you end a command without a semicolon. This means that to run multiple commands at once interactively, you have to format them like this:

```
add x0 x1 x2;
sub x5 x3 0x5;
sub x4 x3 x0
```

Note that the moment we do not end with a semicolon, that marks the end of the testcase.

The `testcases` subdirectory contains test files. If you want to write your own test files, look at the examples in here. We also provide `make` targets `test_week2`, `test_week3`, and `test_week4` to run all the tests for the relevant week.

8.3.5 Debugging your code

Use the following utilities to help debug your code.

- Any compilation errors and warnings will be reported to you by `gcc`, and should be easily solved with its help. We have configured `gcc` so that it will not produce an executable until you have successfully gotten rid of all compilation errors and warnings.
- Any runtime errors that cause your program to crash must be debugged with `gdb`. ***Get out of the habit of using `print` statements for debugging!*** In order to be able to use `gdb` effectively, you will need to compile the code with debug flags enabled. Do this using the command `make debug`. (You may need to run `make clean` before this.) By doing this, the executable will provide additional information to `gdb`, which will allow it to tell you exactly the file and the line within it where the program crashed. Invest the time in learning the capabilities of `gdb`. Asking ChatGPT won't cut it!
- Finally, use the tool `valgrind` (by running `valgrind <executable> <arguments>` from the Linux prompt) to help you find any memory leaks. It will tell you if you have any memory leaks, and with some extra options, information about what is causing them — See the GDB guide on Ed Discussion for more details.

9 Week 2–4: Handing in Your Work

You will hand in these checkpoints using Gradescope for all three weeks. For code, link your GitHub account to Gradescope, and select the `CI-Lab` repository when submitting the assignment. Make sure all files are up to date (i.e., committed) on GitHub before you hand in the assignment.

10 Week 2–4: Evaluation

Each checkpoint of weeks 2-4 is worth 4% of the total course grade, for a total of 13%. Grading will be done by the autograder on Gradescope. Remember that you can upload your work as many times as you want to Gradescope.

Appendices

A Linux Background

This section contains a bit of background on some of the tools and utilities that you will use in the class. If you're already familiar with Linux, you may be able to skip this section; however, we still recommend reading through. *If you get stuck anywhere, please make sure you've read this section before asking in office hours or on Ed Discussion.* If something in this section needs clarification, you of course should still ask.

A.1 Files and Paths in Linux

The terminal is a useful way for navigating the Linux filesystem. To start, there are two kinds of files you will want to know about for this class: *normal files* and *directories*. Normal files are used for pretty much everything — this PDF, images, programs that you will write are all normal files. Directories⁸ are files that contain other files, and are useful for organization.

The filesystem enforces a tree structure, with directories as internal nodes and normal files as leaves. The so-called “root directory” at the top of the tree is always denoted as a slash `/`. Each directory also has a reference to itself and its parent directory, denoted as `.` and `..` respectively. Given the tree structure of the file system, each file also has an associated *absolute path* that starts at the root and traverses each parent directory until the file itself is reached. Each element in the path is separated by a slash `/`. For example, if we have a directory `foo` with the root as its parent, and the file `bar` is contained inside `foo`, then the absolute file path to `bar` is `/foo/bar`.

However, absolute file paths can become long and cumbersome as the number of directories in a file path grows. The terminal also has its own “current working directory”, which is a useful point for starting a *relative* file path. You can see your current working directory at any time with the command `pwd`. Relative paths are essentially the same as absolute paths, but the directories leading up to the current working directory can be omitted from the path. In the example from above, if our terminal's current working directory is `foo`, then the relative path to the file `bar` is just `bar` (or `./bar` if we want to be more specific).

When you first open a terminal, its current working directory will always start in your user's *home directory*, which is often `/home/<your username>/` on a Linux system (though on the UTCS machines this is changed to `/u/<your username>/`). This directory is often abbreviated as `~`, and this character can be used to start absolute file paths as well. The `cd` command allows you to change the current working directory. As a side note, $\text{\LaTeX}$ really doesn't like this `~` character and it tends to change it to a different one when processing the file, so *make sure not to copy-paste commands containing it from this document to your command line, because they won't work*. The correct symbol on an EN-US QWERTY keyboard is the upper one on the leftmost key of the top row of keys, to the left of the `1` key.

All files have permissions associated with them, which specify who is able to access them and what they are able to do with the file. The three kinds of permissions are read, write, and execute, often abbreviated as `rwX`. A user with read permissions for a (normal) file is able to view its contents; a user with write permissions for a (normal) file is able to modify its contents; and a user with execute permissions for a (normal) file is able to run it as an executable (assuming it is actually an executable program). For directories, read permission allows one to list the contents of the directory; write permission allows one to create files or directories within it; and execute permission allows one to change one's current working directory to it. These permissions can be specified for three different sets of users: the file's owner (abbreviated `u`, for

⁸These are called “folders” on Windows and MacOS.

“user”), the owner’s group (abbreviated `g`, for “group”), and every user on the system (abbreviated `o`, for “others”). All of the files that you own will be in your home directory or one of its subdirectories. The manual page for the `chmod` command contains more information on permissions and how to set them for files. Manual pages are explained below.

A.2 Commands and Executables in Linux

Linux has many utilities built-in to the command line, and you can also use it to run your own programs. The first token you type at the command line is interpreted as a file path to an executable, which the terminal will try to run. There are certain utilities contained in the `/bin/` directory, and if the file path does not exist, the terminal will attempt to look in this directory for the file to run it. We refer to such files as *commands*; if we tell you to use a command, it is most likely an executable file in this directory. For example, if we needed to copy a file with the `cp` command, we just type `cp` at the command line instead of `/bin/cp`. If you want to run *your own program*, make sure it has executable permissions and type the (absolute or relative) path to it in the terminal.

Anything after the file path is interpreted as an argument to the program being run. Arguments are just strings of characters passed to the program being run, and can mean different things depending on the program. In the example of `cp`, the full syntax of using it is `cp <source> <destination>`, where `source` is the path to a file to be copied, and `destination` is the path of where to copy the file to. Some arguments are prefixed with a dash `-`, and are referred to as flags. For example, `cp` by default only works with normal files and not directories, but we are able to provide the flag `-r` (for “recursive”) to enable it copy entire directories instead. The flags you can provide to a program always depend on the program being run, but it is very common for programs to accept a `-h` (for “help”) flag, which lists information about the program.

One of the most important and useful commands is `man <command>`, which allows you to access the Linux manual pages. If you are ever unsure what a command does, or you are having trouble using it, `man` is incredibly helpful. Some other useful commands you will need for the assignment are listed in the *Resources* section, and we recommend skimming the manual pages for them.

B Resources

B.1 Linux commands

Table 2 is a minimal list of essential Linux commands that you need to have at your fingertips. Google is your friend to learn about these commands. You can also use the Linux command `man` to find out details of a Linux command⁹ e.g., `man ls`. You will find that many commands have a plethora of command-line options that influence their behavior.

A real fun way to pick up Linux is through Julia Evans’s *Bite Size Linux* zine.

B.2 Git commands

From Wikipedia¹⁰: “Git is a distributed version-control system for tracking changes in any set of files, originally designed for coordinating work among programmers cooperating on source code during soft-

⁹Note that since `man` is itself a Linux command, `man man` is legitimate and does exactly what you think it should.

¹⁰<https://en.wikipedia.org/wiki/Git>

Table 2: A minimal list of essential Linux commands.

Category	Commands
Information	man, history
Navigation	cd, ls, pwd
Remote connection	ssh, scp, ping, exit
File creation/deletion	touch, mkdir, cp, rm
File manipulation	chmod, mv, tar
File access	grep, stat, cat, less, tail

ware development. Its goals include speed, data integrity, and support for distributed, non-linear workflows (thousands of parallel branches running on different systems).”

We will discuss more about version control and the Git data model and commands over the course of the semester, but the following “magic incantations” should suffice for this assignment.

- To **create** a repository, use the Linux command `git init`.
- To **add and commit** either a new file or a modified file to a repository, use the Linux command `git add` `FILENAME` followed by `git commit -m "USEFUL MESSAGE"`. You can also add every modified file using `git add .`, following the `git add` with a period.
- To **update** a remote repository (i.e., your GitHub repository), use the Linux command `git push`.

Just one final note: GitHub is not the same thing as Git! Again, from Wikipedia¹¹: “GitHub, Inc. is [a] provider of Internet hosting for software development and version control using Git. It offers the distributed version control and source code management (SCM) functionality of Git, plus its own features.” There are alternatives to GitHub, such as *Gitlab* and *Bitbucket*.

B.3 Extra resources

Here is a list of extra resources and documentation for some of the utilities used in this lab.

- Make: <https://www.gnu.org/software/make/manual/make.html>
- GDB: <https://sourceware.org/gdb/documentation/>
- C String Library: <https://en.cppreference.com/w/c/string/byte>

C On the Idiosyncrasies of C

This section highlights some of the differences between C and Java that tend to produce the greatest cognitive dissonance for first-time C programmers.

¹¹<https://en.wikipedia.org/wiki/GitHub>

C.1 GDB: Your Friend And Second Source Of Truth

There are three things that you will find invaluable for this lab:

- This specification, which you are currently (thoroughly) reading.
- The CI reference, which we provide you alongside the starter code.
- GDB, which we will talk about right now.

In Java, when your program crashes, it tells you exactly where and why. In C, that is not the case. Very often you will just get an error and a silent exit. The GDB tool is a valuable tool for both debugging and understanding code execution. More info can be found in a pinned thread on Ed Discussion.

We strongly encourage using GDB whenever you are confused as to why your code is not functioning correctly. Even if you are certain that your code behaves as expected, stepping through it will add an additional sanity check.

In addition, GDB can also clear up confusion as to what is happening within the already-written code. We are aware that this might be a lot of information and that it might be difficult to keep it all straight. To that end, we recommend setting a breakpoint at a function you are unsure of and stepping through line-by-line and printing values. Do so whenever you are not sure of what a particular command expects, why this function returns an error, or to understand the overall code flow in general.

The CI reference executable is very useful to understand what the expected output of a testcase is. You can use an online diffchecker to check the difference between what is outputted by your code versus the reference executable for any tests you're failing by simply copy-pasting the respective inputs. It is run in the same way as the `ci` executable, which executes your code.

C.2 The Price of Power

C is much closer to bare metal than Java, which means it offers the programmer a lot more freedom. One of the ways it does so is through explicit memory management.

In Java, when you instantiate an object using the keyword `new`, Java manages requesting memory for that object and releasing the object's memory once the object is no longer used. In C, that is not the case. If you ask for memory (discussed below), it is your job to free said memory whenever you no longer need it, or else you will have a memory leak.

Asking for memory is done by using functions `malloc`, `calloc`, and `realloc`, all located within the `stdlib.h` header. Read the man pages (type in `man malloc` to the terminal prompt, for instance) to see how they work. We encourage you to mostly stick to using `malloc` and `calloc`. As pointed out earlier, asking for memory means you have to free it, which can be accomplished through the function `free`, which is in the same header. We will use a tool called **valgrind** to test your code for memory leaks, and as such you will need to be mindful of using the above procedures and cleaning up after yourself.